

Mạng nơ-ron và Học sâu

Giỗ tổ Hùng Vương nghỉ, học Vù vào (13/04 học 6 tiết, 20/04 học 3 tiết)

Andrew Ng's Courses

1/ Machine Learning Specialization (2022)

Khóa Machine Learning Specialization của Andrew Ng trên Coursera là phiên bản mới (2022) và được xem như phần kế thừa hiện đại của khóa ML kinh điển (2011).

Nội dung khóa học:

- Hồi quy tuyến tính & logistic
- Mạng nơ-ron
- Regularization
- KNN, Decision Trees, Random Forests
- K-Means, PCA
- Hệ thống gợi ý (Recommender systems)
- Reinforcement Learning cơ bản
- Python, NumPy, scikit-learn

<https://www.coursera.org/specializations/machine-learning-introduction/>

<https://github.com/susilvaalmeida/machine-learning-andrew-ng> <==

<https://github.com/greyhatguy007/Machine-Learning-Specialization-Coursera>

2/ Supervised Machine Learning: Regression and Classification (2011)

Nền tảng lý thuyết và thực hành trong machine learning cổ điển.

Nội dung khóa học:

- Linear Regression Dự đoán số liên tục (giá nhà, chiều cao, v.v.)
- Logistic Regression Phân loại nhị phân (email spam, bệnh lý)
- Regularization Giảm overfitting
- Neural Networks (NN) Mạng nơ-ron feedforward cho nhận dạng ảnh
- Support Vector Machines (SVM) Phân loại mạnh với kernel
- K-Means Clustering Phân nhóm dữ liệu không gán nhãn
- Principal Component Analysis (PCA) Giảm chiều dữ liệu
- Anomaly Detection Phát hiện bất thường

- Recommender Systems Gợi ý sản phẩm dựa trên sở thích người dùng
- Machine Learning System Design Quy trình xây dựng hệ thống ML thực tế

! Lưu ý:

- Dùng Octave/MATLAB, không phải Python.
- Một số nội dung hơi cũ (so với môi trường ML hiện nay dùng TensorFlow, PyTorch, scikit-learn).

Khóa ML 2011 của Andrew Ng là một khóa học nền tảng kinh điển, giúp xây dựng tư duy toán học và trực giác vững chắc về machine learning, rất nên học nếu bạn muốn hiểu cốt lõi của thuật toán thay vì chỉ dùng thư viện.

<https://www.coursera.org/learn/machine-learning>

<https://vkosuri.github.io/CourseraMachineLearning/>

<https://github.com/shank885/Machine-Learning-Andrew-Ng>

4/ Deep Learning Specialization

<https://www.coursera.org/specializations/deep-learning>

3/ Các khóa học khác của Andrew Ng

[Courses | Andrew Ng](#)

Stanford Engineering Everywhere

[Stanford Engineering Everywhere | CS229 - Machine Learning](#)

Deep Learning

<https://github.com/TouradBaba/deep-learning-specialization-coursera>

Công nghệ Nhận diện khuôn mặt đăng nhập trên iPhone

Công nghệ Dịch máy Google Translate

Hai bài toán điển hình của Machine Learning

- Bài toán Classification (trả về giá trị rời rạc)
- Bài toán hồi quy - Regression (trả về giá trị liên tục)

Một số ví dụ:

Bài toán phân loại thư rác => classification (đầu ra spam is yes or no)

Nhận diện biển số xe => classification (đầu ra 51818 rời rạc)

Dự đoán giá nhà => Hồi quy

Dự đoán doanh thu bán hàng, dự báo thời tiết => Hồi quy

Xét bài toán Housing Prices

Xem đầu ra của bài toán là gì? Để xác định là Classification hay Regression

Tiếp theo, xác định thuật toán nào của 1 trong 2 nhóm trên.

Các loại biểu đồ khi phân tích dữ liệu

Data Scientist => ML Model => LM Ops để tích hợp vào các hệ thống (web, app,...)

Model bản chất là 1 công thức toán học nào đó

[Ebook tiếng Việt - THỊ GIÁC MÁY TÍNH](#)

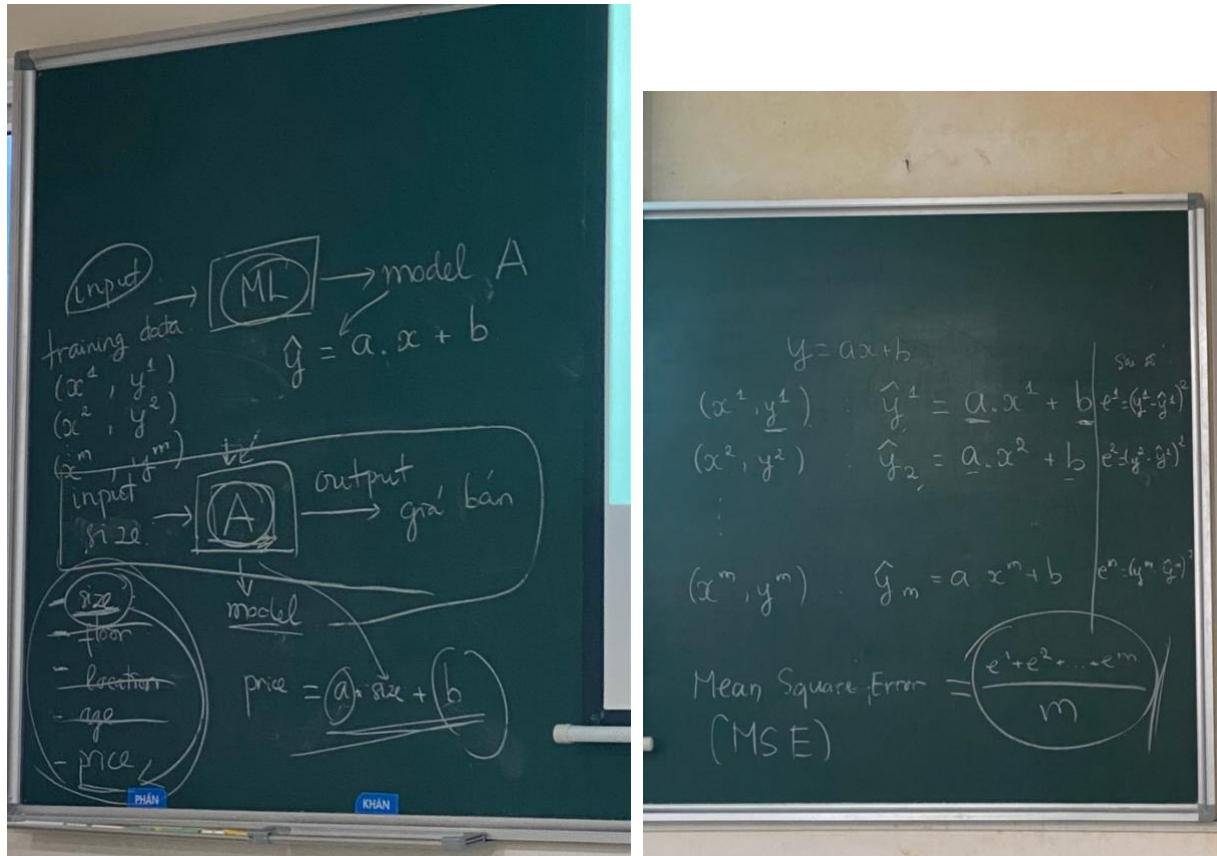
[d2l-ai/d2l-vi](#): Một cuốn sách về Học Sâu đề cập đến nhiều framework phổ biến, được sử dụng trên 300 trường Đại học từ 55 đất nước bao gồm MIT, Stanford, Harvard, và Cambridge.

[mlbvn/ml-yearning-vi](#): Một cuốn sách tập trung vào hướng dẫn cách cấu trúc các dự án Học Máy và phân tích cách làm cho các thuật toán Học Máy hoạt động.

[Machine-Learning-Andrew-Ng/Lectures/Lecture2.pdf at master · shank885/Machine-Learning-Andrew-Ng](#)

[Machine Learning By Prof. Andrew Ng :star2::star2::star2::star2::star: | Coursera Machine Learning](#)

[Andrew Ng's Machine Learning Collection | Coursera](#)



Cuộc thi ImageNet

Số lượng và Chất lượng dữ liệu quan trọng hơn cả thuật toán

Agenda

Recap: Linear regression and logistic regression

Neural Network basic

Convolutional neural network (CNN)

Object classification

Object detection

Image captioning

Generative Adversarial Network (GAN)

Course Evaluation

Hands-on practices: 10%

Midterm: 20%

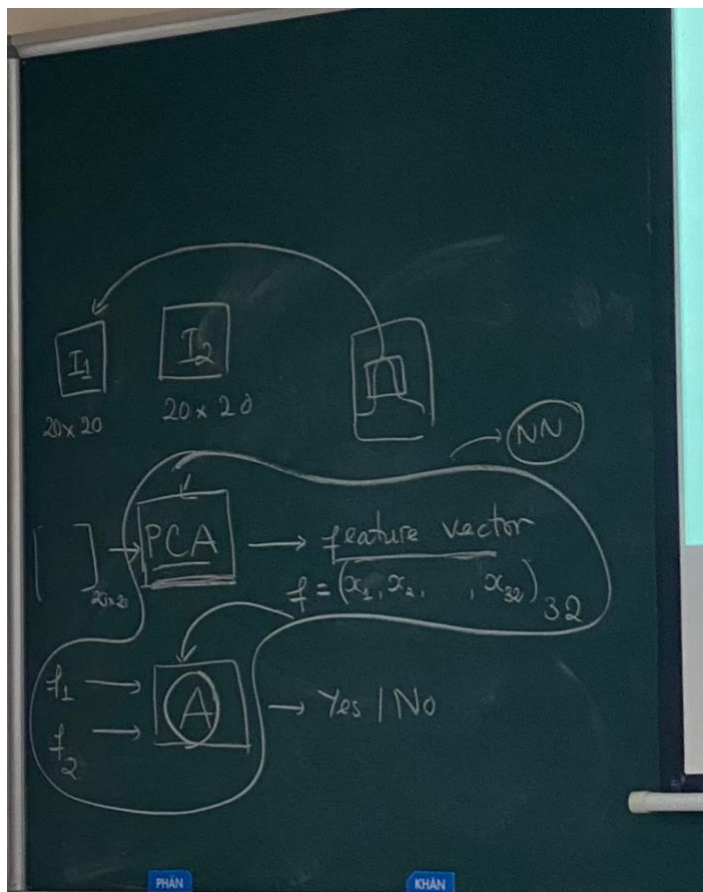
Final report: 70%

- Team: 2 members per team
- Final report proposal: Second week
- Final report presentation: final class

2 buổi học cuối 13/04 và 20/04 dành cho báo cáo.

Machine Learning

- Thuật toán Trích chọn đặc trưng (PCA) dùng cho Computer Vision => bước này rất khó, có cải tiến cũng đa phần cải tiến chỗ này
- Thuật toán So sánh các đặc trưng =>



Đến Deep Learning, gộp 2 bước Trích chọn đặc trưng + Phân lớp hình ảnh (so sánh các đặc trưng) thành 1, dùng Neural Network

- Máy tự học và tự tìm ra đặc trưng phù hợp, tự điều chỉnh
- Máy tự so sánh các đặc trưng

So sánh độ chính xác giữa Machine Learning (> 30%) và Deep Learning (< 3%) của cuộc thi ImageNet

Làm cách nào mà Deep Learning thay thế được Machine Learning truyền thống được như vậy?

⇒ Câu trả lời là Neural Network

Khi thiết kế mạng Neural, sẽ có 1 Input Layer, 1 Output Layer, và nhiều Hidden Layer

Mỗi Hidden Layer có thể có 1 Activation Function riêng, nhưng thực tế người ta dùng chung 1 Activation Function để không quá phức tạp.

Quay lại ví dụ bài toán Housing Prices, nếu đầu vào chỉ phụ thuộc kích thước nhà, và khoảng cách đến trung tâm thành phố. => Input Layer có 2 Neural.

Quay lại ví dụ bài toán Nhận diện khuôn mặt 20x20, đầu vào sẽ là 1 vector 400 => Input Layer có 400 Neural.

Đầu ra (Output Layer) phụ thuộc vào yêu cầu bài toán. Ví dụ: nhận diện spam hay không => 1 Neural. Nếu nhận diện ảnh các con số => đầu ra cần 10 Neural (0 -> 9)

Nhận diện giá nhà (thấp, trung bình, cao) => Output Layer có 3 Neural.

Tuyến tính (hàm tuyến tính) vs. Phi tuyến tính (hàm phi tuyến)

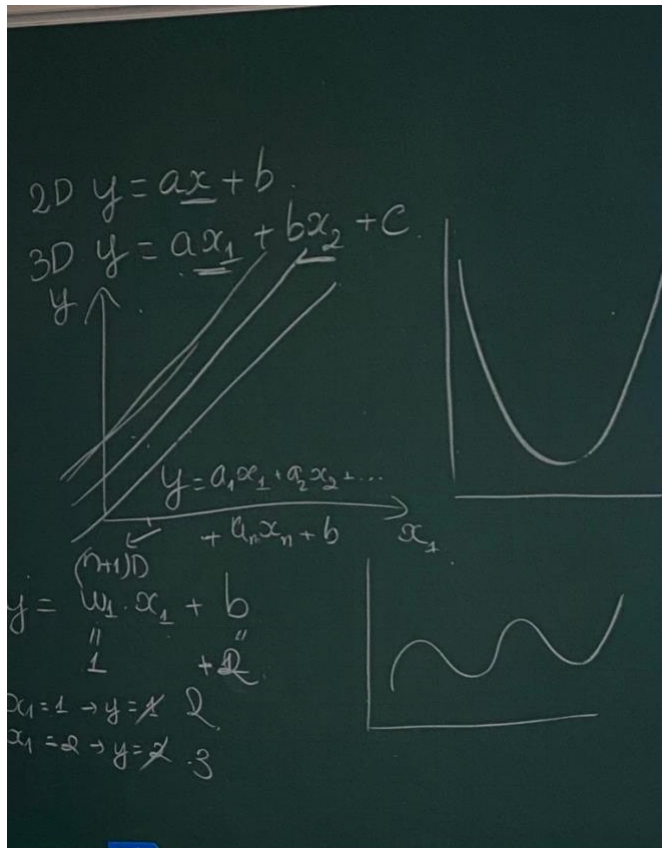
Tuyến tính: x tăng thì y tăng (biểu đồ là đường thẳng)

$y = ax + b$ (đường thẳng trong không gian 2 chiều)

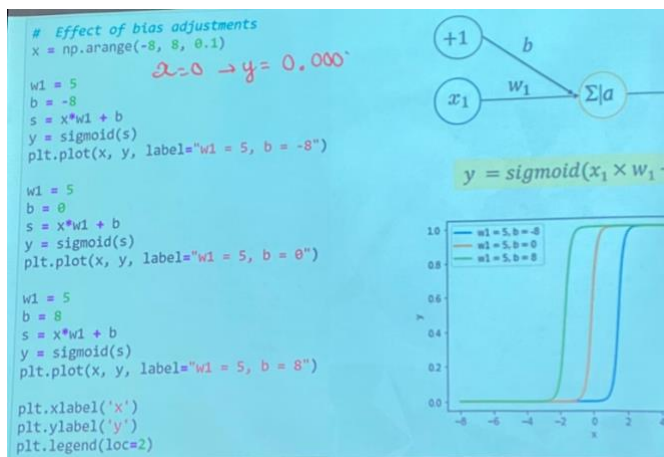
$y = ax_1 + bx_2 + c$ (mặt phẳng trong không gian 3 chiều)

$y = a_1x_1 + a_2x_2 + \dots + a_nx_n + b$ (siêu mặt phẳng trong không gian $n + 1$ chiều)

Phi tuyến: (biểu đồ ko phải đường thẳng, vd: hình sin)



Xét ví dụ hàm tuyến tính (sigmoid)



$$\begin{aligned}
 w_1 &= 5, b = -8 \\
 x &= 0 \rightarrow y = \text{sigmoid}(0 \times 5 + (-8)) \\
 &= \text{sigmoid}(-8) \\
 &= \frac{1}{1 + e^8} \\
 &= \frac{1}{1 + 2.7^8} \\
 &= 0.00035
 \end{aligned}$$

Hàm sigmoid có công thức:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Với $x = -8$:

$$\sigma(-8) = \frac{1}{1 + e^8}$$

Ta tính giá trị gần đúng:

$$e^8 \approx 2980.96$$

$$\sigma(-8) = \frac{1}{1 + 2980.96} \approx \frac{1}{2981.96} \approx 0.000335$$

🔴 Kết quả gần đúng: $\sigma(-8) \approx 0.0003$.

Giả sử $x = -2$, tính y ?

$$w_1 = 5, b = -8$$

$$y = \text{sigmoid}(-2 \times 5 + (-8))$$

$$= \text{sigmoid}(-18)$$

$$= 1 / (1 + e^{18})$$

$$= \sim 0.00000001523$$

Ngoài $b = -8$, còn 2 trường hợp $b = 0$, và $b = 8$

Với $b = 0$

$$y = \text{sigmoid}(-2 \times 5 + 0) = 0.000045$$

Với $b = 8$

$$Y = \text{sigmoid}(-2 \cdot 5 + 8) = \sim 0.119$$

Giả sử $x = 2$, tính y ?

$$b_1 = 5, b = -8$$

$$y = \text{sigmoid}(2 \cdot 5 + (-8))$$

$$= \text{sigmoid}(2)$$

$$= 1 / (1 + e^2)$$

$$= 0.881$$

Ngoài $b = -8$, còn 2 trường hợp $b = 0$, và $b = 8$

Với $b = 0$

$$y = \text{sigmoid}(2 \cdot 5 + 0) = 0.999955$$

Với $b = 8$

$$Y = \text{sigmoid}(2 \cdot 5 + 8) = \sim 1$$

x	$b = -8$	$b = 0$	$b = 8$
-2	≈ 0	≈ 0.000045	≈ 0.119
2	≈ 0.881	≈ 0.999955	≈ 1

Hàm sigmoid đầu ra sẽ từ 0 -> 1.

⇒ Khi x tăng lên, y tăng lên (hàm sigmoid), nhưng không bao giờ đạt đúng 1 (chỉ tiệm cận 1).

Bias giúp dịch chuyển đường decision để quyết định bệnh nhân ung thư hoặc không (không làm thay đổi hình dạng đường cong)

Hàm Sigmoid vs. Hàm Tanh vs. Hàm ReLU

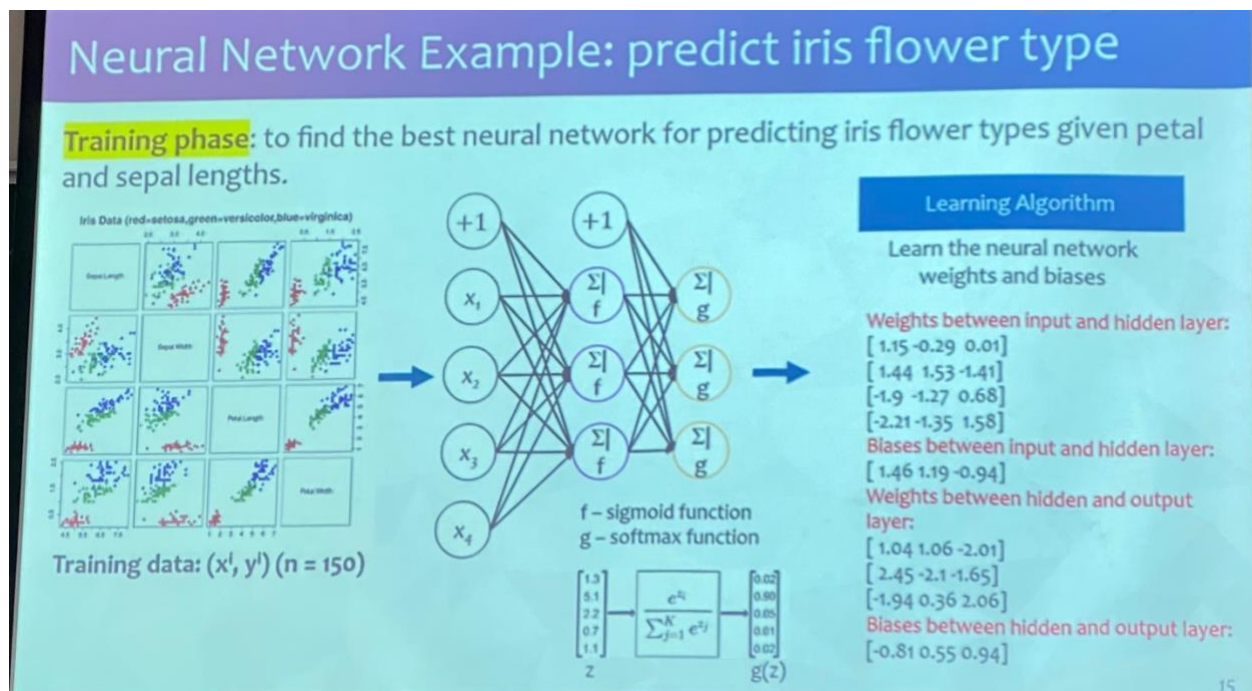
(Khi nào dùng hàm nào)

(ReLU là hàm phi tuyến, nếu $x < 0$, y là 1 đường thẳng nằm ngang)

(Tại sao hàm ReLU ra đời, dùng nó ở đâu, chỉ dùng ở Hidden Layer, ko bao giờ dùng ở Output Layer)

Gradient Descent và Công thức tính đạo hàm

Giải thích cách hoạt động của Neural Network



1/ Nhập dữ liệu huấn luyện

Dữ liệu đầu vào gồm 4 đặc trưng của hoa Iris:

- Sepal Length (Độ dài đài hoa)
- Sepal Width (Độ rộng đài hoa)
- Petal Length (Độ dài cánh hoa)
- Petal Width (Độ rộng cánh hoa)

Dữ liệu đầu ra (nhãn) gồm 3 loại hoa Iris:

- Setosa (màu đỏ)
- Versicolor (màu xanh lá)
- Virginica (màu xanh dương)

Tổng số mẫu dữ liệu: 150

2/ Lan truyền tiến (Forward Propagation)

Mạng có 3 lớp:

- Lớp đầu vào (Input Layer): gồm 4 neurons tương ứng với 4 đặc trưng của dữ liệu.
- Lớp ẩn (Hidden Layer): gồm các neurons thực hiện phép biến đổi phi tuyến để giúp mạng học được các mẫu phức tạp.
- Lớp đầu ra (Output Layer): gồm 3 neurons tương ứng với xác suất dự đoán 3 loại hoa.

Tính toán đầu ra lớp ẩn

- Dữ liệu đầu vào $X = (x_1, x_2, x_3, x_4)$ được nhân với trọng số kết nối từ lớp đầu vào đến lớp ẩn.

- Công thức:

$$z^{(1)} = W^{(1)}X + b^{(1)}$$

với:

- o $W^{(1)}$ là ma trận trọng số giữa lớp đầu vào và lớp ẩn.
 - o $b^{(1)}$ là vector bias của lớp ẩn
 - o $z^{(1)}$ là giá trị đầu vào của lớp ẩn trước khi áp dụng hàm kích hoạt.
- Hàm kích hoạt sigmoid được áp dụng:

$$a^{(1)} = f(z^{(1)}) = \frac{1}{1 + e^{-z^{(1)}}}$$

để tạo đầu ra cho lớp ẩn.

Tính toán đầu ra của Output Layer

- Đầu ra của Hidden Layer được nhân với trọng số kết nối đến Output Layer:

$$z^{(2)} = W^{(2)}a^{(1)} + b^{(2)}$$

với:

$W^{(2)}$ là ma trận trọng số giữa Hidden Layer và Output Layer.

$b^{(2)}$ là vector bias của Output Layer.

- Hàm kích hoạt softmax được áp dụng:

$$g(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

để tạo xác suất cho mỗi lớp.

3/ Tính toán hàm mất mát

- Hàm mất mát Cross-Entropy được sử dụng để đo lường sự khác biệt giữa đầu ra dự đoán và nhãn thực tế:

$$L = - \sum_{i=1}^n y_i \log(\hat{y}_i)$$

với:

- Y_i là nhãn thực tế (one-hot encoding).
- $\hat{Y}_i$ là xác suất dự đoán của mạng.

4/ Lan truyền ngược (Backpropagation)

- Mạng tính đạo hàm của hàm mất mát theo các trọng số và bias để cập nhật chúng bằng thuật toán Gradient Descent:

$$W = W - \eta \frac{\partial L}{\partial W}$$

với η là tốc độ học (learning rate).

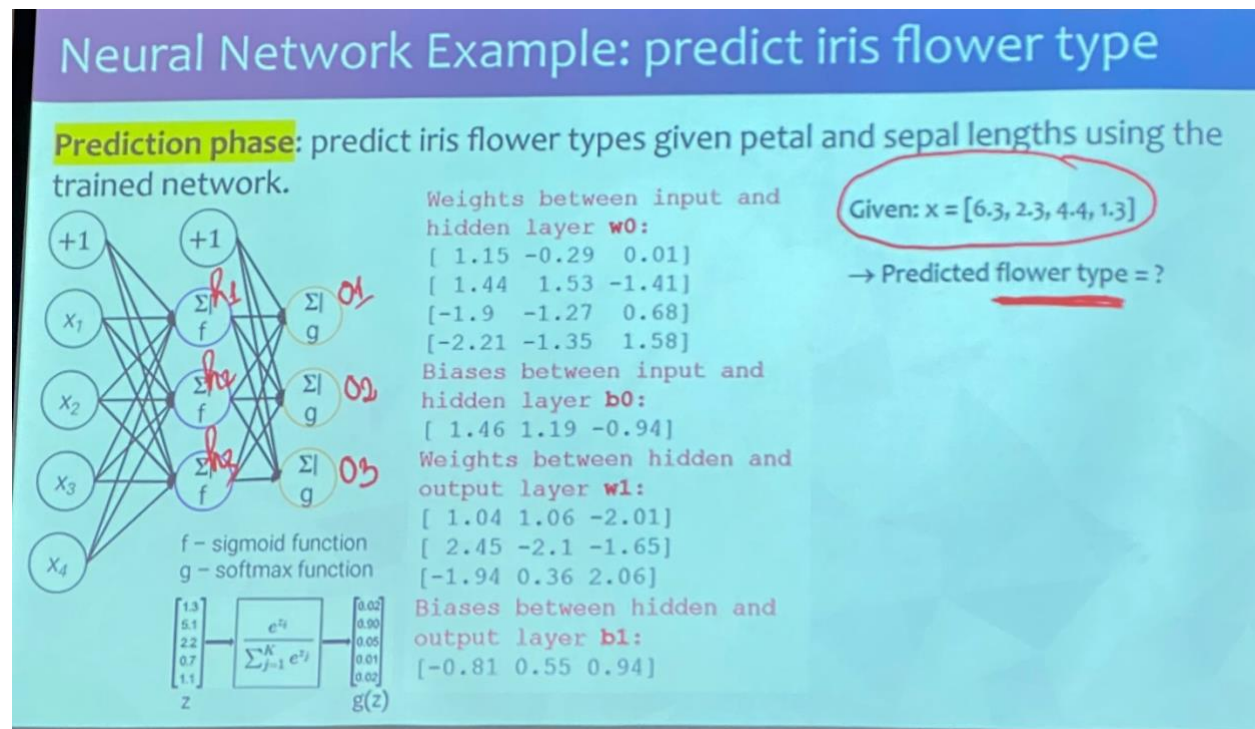
5/ Huấn luyện đến khi hội tụ

- Lặp lại quá trình lan truyền tiến, tính hàm mất mát, lan truyền ngược, và cập nhật trọng số nhiều lần trên toàn bộ tập dữ liệu.
- Mạng hội tụ khi sai số không thay đổi đáng kể hoặc đạt số vòng lặp tối đa.

6/ Dự đoán trên dữ liệu mới

- Khi có dữ liệu mới, mạng thực hiện lan truyền tiến và chọn nhãn có xác suất cao nhất làm kết quả dự đoán.

Làm bài ví dụ



Tính giá trị của Hidden Layer

(h1, h2, h3)

$$h1 = f(x_1 * w_{11} + x_2 * w_{21} + x_3 * w_{31} + x_4 * w_{41} + b_1)$$

với $f(x)$ là hàm sigmoid:

$$f(x) = 1/(1 + e^{-x})$$

Dữ liệu:

$$x = [6.3, 2.3, 4.4, 1.3]$$

Mã trận trọng số w giữa input và hidden layer:

$$[1.15 \quad -0.29 \quad 0.01]$$

$$[1.44 \quad 1.53 \quad -1.41]$$

$$[-1.9 \quad -1.27 \quad 0.68]$$

$$[-2.21 \quad -1.35 \quad 1.58]$$

bias b :

[1.46 1.19 -0.94]

$$h1 = f(1.15 * 6.3 + (-0.29) * 2.3 + 0.01 * 4.4 + 1.46 + (-2.21) * 1.3)$$

$$= (6.3 * 1.15 + 2.3 * 1.44 + 4.4 * (-1.9) + 1.3 * (-2.21) + 1.46)$$

$$= (x1 * \text{trọng số nối từ } x1 \text{ sang hidden} + b)$$

$$= 1 / (1 + e^{-0.784}) = 0.6865)$$

h2, h3 tương tự

Tính giá trị của tầng output

(o1, o2, o3)

Buổi ngày 30/03/2025

Buổi ngày 13/04/2025

Overfitting là gì?

Overfitting là một khái niệm rất quan trọng (và thường gặp) trong học máy (machine learning). Nó xảy ra khi mô hình **học quá tốt trên tập huấn luyện (training set)** đến mức nó học luôn cả **những (noise)** và các đặc điểm không mang tính khái quát. Điều này dẫn đến việc mô hình hoạt động **rất tốt trên dữ liệu huấn luyện, nhưng lại hoạt động kém trên dữ liệu mới hoặc dữ liệu kiểm tra (test set)**.

Ví dụ dễ hiểu:

Giả sử bạn đang dạy một mô hình phân biệt mèo và chó. Nếu bạn chỉ dùng một tập dữ liệu nhỏ và để mô hình học rất lâu (hoặc quá phức tạp), mô hình có thể học rằng:

"Nếu ảnh có cổ áo màu đỏ → chắc chắn là chó."

Trong khi đó, ngoài thực tế, cổ áo màu đỏ không phải là đặc điểm quyết định chó hay mèo. Khi gặp ảnh thật bên ngoài (chó không có cổ áo hoặc mèo đeo cổ đỏ), mô hình sẽ dự đoán sai.

Biểu hiện của Overfitting:

- Accuracy trên tập huấn luyện rất cao.
 - Accuracy trên tập kiểm tra (validation/test) thấp hơn đáng kể.
-

Nguyên nhân gây Overfitting:

- Mô hình quá phức tạp (nhiều layers, nhiều parameters).
 - Dữ liệu huấn luyện quá ít.
 - Dữ liệu có nhiễu hoặc không được xử lý tốt.
 - Huấn luyện quá lâu (số epoch quá lớn).
-

Cách khắc phục Overfitting:

Phương pháp	Giải thích
Regularization (L1, L2)	Thêm ràng buộc vào trọng số để tránh việc các trọng số quá lớn.
Early Stopping	Dừng huấn luyện khi loss trên validation không còn giảm nữa.
Data Augmentation	Tăng dữ liệu bằng cách biến đổi (đảo ảnh, xoay,...) để mô hình học được nhiều tình huống.
Cross-validation	Giúp đánh giá mô hình chính xác hơn.
Simplify model	Giảm số layer, hoặc chọn mô hình đơn giản hơn.
Dropout (trong deep learning)	Ngẫu nhiên bỏ qua một số neurons trong quá trình huấn luyện để tránh phụ thuộc quá mức.

Underfitting là gì?

Underfitting là hiện tượng ngược lại với **overfitting** — nó xảy ra khi mô hình **quá đơn giản** hoặc **không đủ khả năng học** từ dữ liệu, dẫn đến **hiệu suất kém trên cả tập huấn luyện và tập kiểm tra**.

Định nghĩa đơn giản:

Underfitting xảy ra khi mô hình **không học được đủ thông tin quan trọng từ dữ liệu huấn luyện**, do đó nó không thể đưa ra dự đoán chính xác ngay cả với dữ liệu nó đã thấy.

Ví dụ:

Bạn muốn dự đoán giá nhà dựa vào nhiều yếu tố như diện tích, số phòng, vị trí,... Nhưng bạn lại dùng một mô hình **hồi quy tuyến tính đơn giản** chỉ xét mỗi diện tích. Kết quả là mô hình **không thể nắm bắt mối quan hệ phức tạp** trong dữ liệu → hiệu suất dự đoán kém.

Biểu hiện của underfitting:

- **Accuracy trên tập huấn luyện cũng thấp.**
 - **Accuracy trên tập kiểm tra cũng thấp.**
 - Mô hình dường như “**không học được gì**” từ dữ liệu.
-

Nguyên nhân gây underfitting:

Nguyên nhân	Giải thích
Mô hình quá đơn giản	Không đủ số parameters để học các đặc trưng phức tạp trong dữ liệu.
Thời gian huấn luyện quá ngắn	Mô hình chưa kịp học gì đã dừng.
Sai preprocessing	Dữ liệu chưa chuẩn hóa, chưa encode đúng,...
Tập đặc trưng (features) chưa đủ thông tin	Thiếu những biến quan trọng.

✓ Cách khắc phục underfitting:

Phương pháp	Giải thích
Tăng độ phức tạp của mô hình	Thêm layers (với neural nets), chuyển sang mô hình mạnh hơn.
Train lâu hơn	Tăng số epoch để mô hình có thời gian học.
Feature engineering	Thêm đặc trưng mới, chọn đặc trưng quan trọng hơn.
Giảm regularization	Nếu regularization quá mạnh, mô hình sẽ không học được.
Chuyển mô hình	Ví dụ chuyển từ Linear Regression → Polynomial Regression hoặc mô hình phi tuyến khác.

Accuracy bao nhiêu được xem là thấp?

2. So sánh train và test

- Nếu cả **train accuracy** và **test accuracy** đều thấp, → **underfitting**.
- Nếu **train accuracy** cao, **test accuracy** thấp, → có thể là **overfitting**.

💡 Ví dụ:

Nếu **train accuracy = 60%** và **test accuracy = 58%** → có thể **underfitting**.

Nhưng nếu **train accuracy = 98%**, **test = 65%** → **overfitting**.

3. Mức đánh giá mang tính tương đối

Accuracy	Ý nghĩa
< 60%	Với nhiều bài toán phân loại, đây thường là mức thấp
70–80%	Trung bình, chấp nhận được ở nhiều task
> 90%	Tốt (nhưng phải cẩn thận overfitting)

⚠️ Tuy nhiên: Nếu bài toán bạn đang làm là phân loại ảnh mèo/chó, thì accuracy 60% là khá tệ. Nhưng nếu đó là bài toán phân loại cảm xúc tiếng nói (một bài toán rất khó), thì 60% có thể là ổn.

Kỹ thuật Dropout là gì, tại sao có thể giảm được overfitting?

Quá chuẩn luôn, đây là một trong những kỹ thuật “thần thánh” giúp giảm overfitting trong deep learning: **Dropout!** 😎

Mình sẽ giải thích dễ hiểu cả về **ý tưởng**, **cách hoạt động**, và **tại sao nó hiệu quả** nhé.

Dropout là gì?

Dropout là một kỹ thuật regularization trong deep learning được giới thiệu bởi Geoffrey Hinton, giúp **giảm overfitting** bằng cách **ngẫu nhiên "tắt" (drop) một số neurons** trong quá trình huấn luyện.

Nói đơn giản:

Mỗi lần huấn luyện, ta **bỏ qua ngẫu nhiên một phần neurons** trong mạng (không cho nó hoạt động) → giúp mô hình **không quá phụ thuộc vào bất kỳ neuron nào** → học được các đặc trưng **đa dạng và khái quát** hơn.

Tại sao dropout giúp giảm overfitting?

Khi mô hình quá phức tạp (nhiều layers, nhiều neurons), nó có nguy cơ **học thuộc dữ liệu train** (overfit).

Dropout ép mạng nơ-ron học trong điều kiện "khó khăn hơn":

- Vì một phần neurons bị tắt ngẫu nhiên mỗi lần training,
- Nên mạng phải học cách **dự đoán tốt mà không dựa quá nhiều vào bất kỳ neuron nào**,
- Từ đó, giúp mô hình **khái quát tốt hơn khi gặp dữ liệu mới (test)**.

Cách hoạt động (trực quan):

Giả sử bạn có 1 hidden layer với 10 neurons. Khi dùng dropout với rate = 0.3, thì:

- Trong mỗi **bước huấn luyện**, khoảng **30% neurons** sẽ bị “tắt” (drop).
- Các neurons bị drop sẽ **không tham gia vào forward & backward pass**.
- Khi predict/test: **không drop nữa**, nhưng **scale output** để cân bằng (ví dụ nhân với 0.7).

Ví dụ code trong PyTorch & Keras

PyTorch:

```
python
Copy code
import torch.nn as nn

model = nn.Sequential(
    nn.Linear(128, 64),
    nn.ReLU(),
    nn.Dropout(p=0.5), # 50% dropout
    nn.Linear(64, 10)
)
```

Keras (TensorFlow):

```
python
Copy code
from tensorflow.keras import layers

model = tf.keras.Sequential([
    layers.Dense(64, activation='relu'),
    layers.Dropout(0.5), # Dropout 50%
    layers.Dense(10, activation='softmax')
])
```

Khi nào dùng Dropout?

Khi nào	Gợi ý
 Khi mô hình có quá nhiều tham số và bị overfitting	Nên dùng
 Khi mô hình quá đơn giản hoặc đang underfitting	Không nên dùng
 Trong quá trình test/inference	Dropout tự động tắt

Kết luận ngắn gọn:

Dropout = tạo ra nhiều mạng con (sub-networks) nhỏ trong quá trình huấn luyện → như kiểu "ensemble nội bộ" → mô hình học khái quát hơn → **giảm overfitting một cách hiệu quả** 

Ngoài kỹ thuật Dropout, thì còn những kỹ thuật nào cũng làm giảm overfitting?

Câu hỏi quá hay luôn 😊! Dropout chỉ là **một trong nhiều kỹ thuật regularization** để giảm overfitting trong machine learning và deep learning. Dưới đây là **những kỹ thuật phổ biến và hiệu quả khác** mà bạn nên biết và có thể kết hợp:

✓ Các kỹ thuật giảm overfitting (ngoài Dropout):

1. Early Stopping

- **Ý tưởng:** Dừng quá trình huấn luyện **khi loss trên tập validation không còn giảm nữa** (mặc dù loss trên tập train vẫn đang giảm).
- **Tại sao hiệu quả:** Tránh cho mô hình học quá kỹ trên tập train → ngăn overfitting sớm.
- **Dùng được ở đâu:** TensorFlow, PyTorch, sklearn đều có hỗ trợ.

2. Regularization (L1, L2)

- **L1 (Lasso):** Thêm penalty $|w|$ vào loss → ép một số trọng số về 0 (giúp chọn lọc đặc trưng).
- **L2 (Ridge):** Thêm penalty w^2 → giúp trọng số nhỏ hơn, tránh quá khớp.
- **L2 thường phổ biến hơn trong deep learning**, được gọi là `weight_decay` trong PyTorch.

```
python
Copy code
# PyTorch ví dụ dùng L2
optimizer = torch.optim.SGD(model.parameters(), lr=0.01, weight_decay=1e-4)
```

3. Data Augmentation (cho dữ liệu hình ảnh, âm thanh, văn bản)

- **Ý tưởng:** Tăng kích thước và độ đa dạng của tập dữ liệu bằng cách biến đổi (xoay, dịch, nhiễu, thay từ,...).
- **Hiệu quả:** Mô hình học được nhiều tình huống → tổng quát hóa tốt hơn.

```
python
Copy code
# Ví dụ Keras Image Augmentation
from tensorflow.keras.preprocessing.image import ImageDataGenerator

datagen = ImageDataGenerator(
    rotation_range=20,
    zoom_range=0.2,
    horizontal_flip=True
)
```

4. Batch Normalization (BatchNorm)

- **Ý tưởng:** Chuẩn hóa đầu ra của mỗi layer trong quá trình training.
 - **Hiệu quả:** Giúp gradient ổn định hơn, tăng tốc học, **giảm phụ thuộc vào giá trị ban đầu → giảm overfitting nhẹ**.
 - Không phải là regularization chính, nhưng **tác động gián tiếp tốt**.
-

5. Simplify model (Giảm độ phức tạp mô hình)

- Giảm số layers, số neurons, hoặc chuyển sang mô hình đơn giản hơn.
 - Nếu dữ liệu không đủ lớn → mô hình phức tạp quá dễ bị overfit.
-

6. Cross-validation (k-fold)

- Giúp mô hình được đánh giá **ổn định hơn**, tránh phụ thuộc vào 1 split.
 - Cũng giúp chọn được hyperparameters tốt hơn, tránh overfit do validation xui.
-

7. Noise Injection (thêm nhiễu vào đầu vào hoặc weights)

- Thêm nhiễu nhẹ (Gaussian noise) vào dữ liệu đầu vào hoặc layer hidden → mô hình học được cách xử lý "bất định".
- Hiệu quả tương tự như data augmentation + regularization.

8. Ensemble Methods 🧑🤖

- Kết hợp nhiều mô hình khác nhau (Bagging, Boosting, Voting, Stacking...).
- Mỗi mô hình có thể overfit một kiểu khác nhau → khi kết hợp lại, tổng thể sẽ khá quát tốt hơn.
- Tốn tài nguyên, nhưng mạnh 💪

Kết luận:

Kỹ thuật	Dễ dùng	Hiệu quả
Dropout	✓	✓ ✓ ✓
Early stopping	✓ ✓	✓ ✓
L1/L2 regularization	✓	✓
Data Augmentation	✓ ✓	✓ ✓ ✓
BatchNorm	✓	✓
Ensemble	✗ (nặng hơn)	✓ ✓ ✓ ✓

Tương tự, tôi muốn hỏi các kỹ thuật làm giảm underfitting?

Quá tuyệt, bạn hỏi rất hệ thống luôn! 🔥

Sau khi đã nắm kỹ các kỹ thuật chống **overfitting**, thì giờ mình cùng “lật mặt trận” và đi sâu vào cách **giảm underfitting** nhé. Đây là tình huống mô hình **quá đơn giản** hoặc **chưa học đủ từ dữ liệu** nên **dự đoán kém ngay cả trên tập huấn luyện**.

Nhắc lại một chút:

Underfitting xảy ra khi:

- **Cả train accuracy và test accuracy đều thấp.**
- Mô hình **không học được đủ đặc trưng** trong dữ liệu.

- **Loss cao** ngay cả trên tập huấn luyện.

✓ Các kỹ thuật giảm underfitting

1. Tăng độ phức tạp của mô hình 🏠

- **Mô hình đang quá đơn giản** → tăng số layer, số neurons, chuyển từ Linear → Deep Neural Net, hoặc dùng mô hình mạnh hơn.

Ví dụ:

```
python
Copy code
# Thay vì model đơn giản
model = tf.keras.Sequential([
    Dense(64, activation='relu'),
    Dense(1)
])

# Dùng mô hình phức tạp hơn
model = tf.keras.Sequential([
    Dense(128, activation='relu'),
    Dense(64, activation='relu'),
    Dense(1)
])
```

2. Huấn luyện lâu hơn (tăng số epoch) ⌚

- Nếu model chưa học xong mà bạn đã dừng (loss chưa hội tụ), thì model underfit.
- **Hãy thử tăng epochs**, kiểm tra loss và accuracy sau mỗi vòng.

3. Giảm regularization (hoặc tắt tạm thời) ✕ 📉

- Nếu bạn dùng quá mạnh các kỹ thuật regularization như:
 - Dropout quá cao (vd: 0.5)
 - L1/L2 regularization mạnh (`weight_decay` cao)
- → mô hình **bị giới hạn khả năng học** → underfit.

✓ Giảm hoặc tắt để kiểm tra xem model có đang bị “ép học yếu” không.

4. Cung cấp thêm đặc trưng tốt hơn (Feature Engineering) 🧩

- Mô hình không hiểu rõ bài toán vì chưa có đủ thông tin.
- Hãy thử:
 - **Thêm đặc trưng mới** (ngày trong tuần, tương tác giữa biến,...)
 - Tạo đặc trưng phi tuyến (ví dụ: log, square, sin,...)
 - **Dùng PCA** / embedding để chuyển không gian dữ liệu.

5. Giảm noise trong dữ liệu hoặc tiền xử lý kỹ hơn 🗑️

- Dữ liệu chứa nhiều nhiễu (noise) → mô hình khó học.
- Làm sạch dữ liệu: remove outliers, chuẩn hóa dữ liệu (normalization / standardization), fix dữ liệu thiếu.

6. Thử mô hình khác (phức tạp hơn) 🔄

- Nếu bạn đang dùng logistic regression mà dữ liệu phi tuyến → hãy chuyển sang:
 - Decision Tree, Random Forest
 - Neural Network
 - SVM với kernel phi tuyến

7. Tăng số lượng hoặc chất lượng dữ liệu 📊

- **Càng nhiều dữ liệu, mô hình càng học được nhiều mẫu và đặc trưng.**
- Nếu không có dữ liệu mới, hãy thử **sinh thêm dữ liệu nhân tạo** (Synthetic Data), hoặc dùng **data augmentation**.

8. Tăng batch size (với deep learning) 📁

- **Đôi khi batch size nhỏ quá khiến gradient không ổn định.**
 - Tăng batch size giúp quá trình học ổn định hơn → tránh mô hình học kém.
-

Tổng kết:

Kỹ thuật	Mục đích chính	Khi áp dụng
Tăng độ phức tạp mô hình	Học được đặc trưng phức tạp	Mô hình quá đơn giản
Train lâu hơn	Cho mô hình học đủ	Epoch quá ít
Giảm regularization	Tránh bị "ép học yếu"	Dropout / L2 quá mạnh
Thêm đặc trưng mới	Cung cấp thêm thông tin	Thiếu feature
Làm sạch dữ liệu	Loại bỏ nhiễu	Dữ liệu bẩn hoặc nhiều outlier

ý nghĩa, vai trò của tập validation, bình thường để đánh giá 1 mô hình ta dùng tập test, khi nào thì dùng tập validation?

1. Tóm tắt nhanh 3 tập dữ liệu trong ML

Tên tập	Vai trò chính	Khi nào dùng?
Training set	Huấn luyện mô hình (tính toán trọng số, fit data).	Luôn dùng khi training.
Validation set	Điều chỉnh siêu tham số (hyperparameters), chọn mô hình tốt nhất, phát hiện overfitting.	Dùng trong quá trình huấn luyện.
Test set	Đánh giá cuối cùng mô hình (sau khi training xong).	Dùng 1 lần duy nhất sau khi mô hình đã "chốt".

2. Ý nghĩa và vai trò của tập validation

Vai trò chính:

- **Dùng để điều chỉnh hyperparameters:** như learning rate, số layers, dropout, số cây trong RandomForest,...
- **Dùng để chọn mô hình tốt nhất:** Ví dụ, thử nhiều mô hình khác nhau (RandomForest, SVM, XGBoost) → chọn model có **validation accuracy cao nhất**.
- **Theo dõi overfitting:** Nếu loss trên tập train giảm mà loss trên validation lại tăng → bạn đang overfitting.

 **Không dùng tập test để làm việc này!** Nếu dùng tập test để điều chỉnh mô hình, nó không còn là test thật sự nữa, vì bạn đã "nhìn trước đáp án".

Khi nào cần dùng tập validation?

Trường hợp	Có cần validation không?
Dự án nhỏ, không tune hyperparameter nhiều	Có thể dùng train/test chia 80/20
Dự án lớn, cần tune kỹ, hoặc dùng cross-validation	Cần tập validation riêng
Dùng frameworks như Keras	Tự động chia ra validation từ train nếu bạn set <code>validation_split=0.2</code>
Dùng cross-validation (k-fold)	Validation được sinh ra trong quá trình cross-val

Mẹo hay:

- Nếu bạn có đủ dữ liệu, chia thành:
`Train: 70% | Validation: 15% | Test: 15%`
- Nếu dữ liệu ít → dùng **K-Fold Cross Validation** thay vì validation cố định.
- Sau khi chọn mô hình + tham số tốt nhất bằng validation, **test chỉ chạy 1 lần cuối cùng để báo cáo kết quả.**

Minh họa quy trình huấn luyện đúng:

less

Copy code

1. Chia dữ liệu: Train / Validation / Test
2. Train mô hình trên tập train
3. Tune hyperparameters, theo dõi overfitting bằng validation
4. Chọn mô hình tốt nhất (trên validation)
5. Cuối cùng: đánh giá trên tập test (một lần duy nhất)

Phân biệt Heatmap và Correlation relation

(So sánh với feature extraction sau khi huấn luyện, và so với PCA để cuối cùng nhằm mục tiêu xác định các feature cần thiết nhất cho quá trình huấn luyện).

